

Reviewer Pack — Symmetric Tensor Rank Gap in Permanent vs Determinant Decomp...

Entry a65db1164c92 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 23:33:26 UTC

Symmetric Tensor Rank Gap in Permanent vs Determinant Decompositions

Entry ID: a65db1164c92 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 23:33:26 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Groups (Schur-Weyl Duality)

Field B (complexity object): Boolean Circuit Size for Permanent Polynomial

Statement:

For any n -variable polynomial f , the symmetric tensor rank of its decomposition under the symmetric group action is $\Omega(n^2)$ for the permanent and $O(n)$ for the determinant. Thus, the circuit size of f is at least $\Omega(n^2)$ if f is the permanent.

Rationale (proposer's reasoning):

The symmetric tensor rank captures the complexity of decomposing the polynomial into symmetric components. The permanent's higher rank suggests it requires more resources to compute, aligning with known lower bounds.

Taxonomy category: GCT_DET_PERM (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4469b4d56528f701

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.4s

5.1 Generated Python source

```
import random
import math
import sys

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below the pivot
        factor = 1 / A[i][i]
        for j in range(i, n):
            A[i][j] *= factor
        for k in range(i+1, n):
            factor = A[k][i]
            for j in range(i, n):
                A[k][j] -= factor * A[i][j]
    return A

def rank(A):
    A = gaussian_elimination(A)
    rank = 0
    for row in A:
        if any(row):
            rank += 1
    return rank

def matrix_multiplication(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C
```

```

def create_symmetric_tensor(n, clauses):
    tensor = [[0] * n for _ in range(n)]
    for clause in clauses:
        for i in clause:
            for j in clause:
                tensor[i-1][j-1] += 1
    return tensor

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    num_clauses = random.randint(n, n * (n - 1))

    # Generate a random CNF formula
    clauses = []
    for _ in range(num_clauses):
        clause = set(random.sample(range(1, n + 1), random.randint(1, n)))
        clauses.append(clause)

    # Convert to symmetric tensor
    A = create_symmetric_tensor(n, clauses)

    # Compute symmetric tensor rank
    tensor_rank = rank(A)

    # The conjecture states that the permanent's rank is  $\Omega(n^2)$  and determinant's rank is  $O(n)$ 
    # For simplicity, we will check if the rank is at least  $n^2$  for a random permutation of variables
    permuted_clauses = [set(random.sample(range(1, n + 1), len(clause))) for clause in clauses]
    permuted_A = create_symmetric_tensor(n, permuted_clauses)
    permuted_rank = rank(permuted_A)

    # Check if the rank is at least  $n^2$ 
    conjecture_holds = permuted_rank >= n**2

    return {
        "metric_name": "symmetric_tensor_rank",
        "metric_value": tensor_rank,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "rank <  $n^2$ "
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_rank)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

```

```

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_rank} std={std_dev} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"rank < n^2\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

name': 'symmetric_tensor_rank', 'metric_value': 27, 'instances_tested': 1, 'conjecture_holds': False
TRIAL: {'metric_name': 'symmetric_tensor_rank', 'metric_value': 21, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'symmetric_tensor_rank', 'metric_value': 18, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'symmetric_tensor_rank', 'metric_value': 15, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'symmetric_tensor_rank', 'metric_value': 7, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'symmetric_tensor_rank', 'metric_value': 25, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'symmetric_tensor_rank', 'metric_value': 23, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'symmetric_tensor_rank', 'metric_value': 9, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'symmetric_tensor_rank', 'metric_value': 38, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'symmetric_tensor_rank', 'metric_value': 16, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'symmetric_tensor_rank', 'metric_value': 36, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'symmetric_tensor_rank', 'metric_value': 19, 'instances_tested': 1, 'conjecture_holds': False}
TRIAL: {'metric_name': 'symmetric_tensor_rank', 'metric_value': 17, 'instances_tested': 1, 'conjecture_holds': False}
RESULT: FALSIFIED counterexample="rank < n^2" first_failing_seed=11

```

8. Critic adversarial review

Critic verdict: CHALLENGE

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

parse_fail | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	92729
2	propose	ollama_remote	qwen3:8b	0	0	99845
3	preregistration	ollama_remote	qwen3:8b	0	0	24993
4	novelty	ollama_remote	qwen3:8b	0	0	20717
5	novelty	ollama_remote	qwen3:8b	0	0	16758
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12015
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10797
8	critic	ollama_remote	qwen3:8b	0	0	29895
9	judge	ollama_remote	qwen3:8b	0	0	21828

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 329577 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/a65db1164c92.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/a65db1164c92.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/a65db1164c92.tar.gz (if generated)